

SAMVAAD AI

Hospital AI Voice Booking Platform

PROJECT HANDOVER DOCUMENT

Version	1.0.0
Date	June 23, 2026
Status	Production
Classification	Confidential

Revision History

Version	Date	Author	Changes
1.0.0	2026-06-23	Technical Team	Initial handover document

1. Executive Summary

Project Name

Samvaad AI (package: samvaad-server)

Purpose

Samvaad AI is a full-stack Hospital Management and AI Voice Booking Agent platform. It enables hospitals to manage patients, doctors, appointments, and prescriptions through a REST API, and automates patient appointment booking via an AI-powered voice agent that answers SIP phone calls in Hindi and English.

Business Objective

- Reduce hospital receptionist workload by automating appointment booking via voice AI
- Deliver post-booking confirmation, prescription reminders, and medicine reminders via WhatsApp
- Provide hospitals with a unified dashboard for patient records, revenue, and analytics
- Enable automated payment collection and payout management through Razorpay

Key Features

Feature	Description
AI Voice Booking Agent	LiveKit + OpenAI Realtime API; books appointments in Hindi & English via SIP phone call
WhatsApp Notifications	Appointment confirmations, prescriptions, medicine reminders via Meta Cloud API
Appointment Management	Full CRUD with slot availability, doctor schedules, and Google Calendar integration
Medicine Reminders	BullMQ-scheduled WhatsApp reminders at breakfast / lunch / dinner times
Prescription Management	Digital prescriptions with WhatsApp delivery and patient portal deep-link
Payment Integration	Razorpay payment collection with webhook verification and payout tracking
Multi-Hospital Support	Complete data isolation per hospital; role-based access control across hospitals
Call Queue System	Phone hold queue with music, transfer messages, and LiveKit SIP integration
Google Calendar Sync	Doctor appointment sync + Google Meet video link via OAuth 2.0
Exotel Call Analytics	Monthly call log sync and analytics for inbound/outbound calls

Current Status

PRODUCTION — deployed on a Linux VPS (DigitalOcean Droplet) managed by PM2. Four independent PM2 processes serve the API, voice agent, queue worker, and medicine reminder scheduler.

2. System Architecture

High-Level Architecture

[illegible]

Voice Call — Step-by-Step Data Flow

Step	Description
1	Patient calls hospital ExoPhone number (Exotel)
2	Exotel routes SIP call to LiveKit Cloud room
3	Queue Worker plays hold music while agent prepares (if LIVEKIT_QUEUE_ENABLED)
4	LiveKit Worker accepts call; Silero VAD detects when caller is speaking
5	Sarvam STT (batch REST mode) transcribes caller audio to text
6	OpenAI Realtime model processes text; calls tools: list_available_slots → search_doctors → create_appointment
7	Tool results spoken to caller via Sarvam TTS
8	On booking: BullMQ enqueues WhatsApp confirmation job (5 retries, exponential backoff)
9	Post-call: GPT-4.1 extracts unstructured notes and updates appointment record in MongoDB

External Integrations

Service	Role	Protocol
OpenAI Realtime API	Voice agent AI brain (gpt-realtime-mini)	WebSocket
OpenAI GPT-4.1	Post-call data extraction	HTTPS REST
Sarvam AI	Hindi/English STT + TTS	HTTPS REST
LiveKit Cloud	WebRTC / SIP voice infrastructure	WebSocket + SIP

Service	Role	Protocol
Meta WhatsApp Cloud API	Outbound WhatsApp messages	HTTPS REST (Graph API)
Razorpay	Payment collection + webhooks	HTTPS REST
Google Calendar API	Appointment events + Meet links	HTTPS REST (OAuth 2.0)
Exotel	SIP telephony + call analytics	SIP + HTTPS REST
Mailtrap	Transactional email (password reset)	HTTPS REST
Redis	BullMQ job queues	TCP
MongoDB	Primary database	TCP (Mongoose ODM)

3. Technology Stack

Backend Runtime & Framework

Component	Technology	Version
Runtime	Node.js	≥ 18.x
Framework	Express.js	^4.22.1
Database ODM	Mongoose	^8.7.0
Job Queues	BullMQ	^5.72.1
Redis Client	ioredis	^5.10.1
Auth Tokens	jsonwebtoken	^9.0.2
Password Hashing	bcryptjs	^2.4.3
Rate Limiting	express-rate-limit	^8.2.1
Validation	express-validator + zod	^7.3.1 / ^3.25.76
File Upload	multer	^2.0.2
Cron Jobs	node-cron	^3.0.3

AI / Voice Stack

Component	Technology	Version
Voice Agent SDK	@livekit/agents	^1.2.1
OpenAI Plugin	@livekit/agents-plugin-openai	^1.2.1
VAD (Voice Activity Detection)	@livekit/agents-plugin-silero	1.2.2
Noise Cancellation	@livekit/noise-cancellation-node	^0.1.9
LiveKit Server SDK	livekit-server-sdk	^2.15.0
OpenAI SDK	openai	^6.21.0
Sarvam AI SDK	sarvamai	^1.1.7

Frontend

Not Found in Codebase — this repository is the backend API only. A separate frontend (Vite/React at <http://localhost:5173> in development) is referenced in CORS settings but is not part of this repository.

4. Folder Structure

```
samvaad-server/
  ■■■ index.js ← Main entry: starts API + embedded workers
  ■■■ ecosystem.config.cjs ← PM2 process definitions (4 processes)
  ■■■ package.json
  ■
  ■■■ src/ ← API server source
  ■ ■■■ app.js ← Express: CORS, middleware, routes
  ■ ■■■ config/
  ■ ■ ■■■ db.js ← MongoDB connection (connectDB)
  ■ ■ ■■■ env.js ← All environment variables with defaults
  ■ ■■■ constants/roles.js ← ROLES enum + ROLE_HIERARCHY
  ■ ■■■ controllers/ ← Request handlers (one per resource)
  ■ ■■■ middleware/
  ■ ■ ■■■ auth.js ← JWT protect + optionalAuth
  ■ ■ ■■■ roles.js ← requireAdmin, requireStaff, etc.
  ■ ■■■ models/ ← Mongoose schemas (18 collections)
  ■ ■■■ routes/index.js ← Route aggregator (all /api/* prefixes)
  ■ ■■■ services/ ← Business logic + external API clients
  ■ ■ ■■■ appointmentWhatsAppNotify.js
  ■ ■ ■■■ appointmentConfirmationDispatch.js ← BullMQ enqueue helper
  ■ ■ ■■■ reminder.service.js ← Medicine reminder scheduling
  ■ ■ ■■■ whatsappCloud.js ← Meta Cloud API client
  ■ ■■■ queues/ ← BullMQ queue definitions
  ■ ■■■ workers/ ← BullMQ worker processors
  ■ ■ ■■■ reminder.worker.js ← Sends medicine reminders
  ■ ■ ■■■ appointmentConfirmation.worker.js
  ■ ■■■ agent/ ← AI agent support
  ■ ■■■ hospitalPrompt.js ← Dynamic system prompt (IST date/time injected)
  ■ ■■■ realtimeToolHandlers.js ← Tool implementations (book/search/slots)
  ■ ■■■ realtimeTools.js ← Tool schema definitions for OpenAI
  ■
  ■■■ livekit-agent/ ← Voice Agent (separate PM2 process)
  ■ ■■■ main.js ← Agent entry (LiveKit defineAgent)
  ■ ■■■ agent.js ← Core logic + tool result sanitizer
  ■ ■■■ phoneQueueWorker.js ← Hold queue process
  ■ ■■■ sarvamStt.js / sarvamTts.js ← Sarvam AI adapters
  ■ ■■■ endPhoneCall.js ← Auto-hangup after booking
  ■ ■■■ postCallPipeline.js ← GPT-4 post-call extraction
  ■
  ■■■ scripts/import-medicines/ ← Bulk Excel medicine importer
  ■ ■■■ import-medicines.js ← Streaming xlsx importer (300k+ rows)
  ■ ■■■ config.js ← Column mapping (edit for new files)
  ■
  ■■■ docs/ ← Project documentation
```

■■ uploads/ ← Hospital logos + uploaded files

5. Environment Configuration

IMPORTANT: In production, PM2 ecosystem env overrides .env values. To change a production env var, edit .env AND run: pm2 delete && pm2 start ecosystem.config.cjs --only

Core Server

Variable	Purpose	Required
NODE_ENV	production or development	Optional (default: development)
PORT	API HTTP listen port	Optional (default: 3000)
TZ	Server timezone	Optional (default: Asia/Kolkata)
MONGODB_URI	MongoDB connection string	Required
UPLOADS_ROOT	Filesystem path for uploaded files	Optional

Authentication

Variable	Purpose	Required
JWT_ACCESS_SECRET	Signs access tokens — CHANGE IN PRODUCTION	Required
JWT_REFRESH_SECRET	Signs refresh tokens — CHANGE IN PRODUCTION	Required
JWT_ACCESS_EXPIRY	Access token TTL (default: 15m)	Optional
JWT_REFRESH_EXPIRY	Refresh token TTL (default: 7d)	Optional
JWT_PASSWORD_RESET_SECRET	Signs password-reset tokens	Optional

WhatsApp (Meta Cloud API)

Variable	Purpose	Required
WHATSAPP_CLOUD_ACCESS_TOKEN	Meta permanent access token	Required
WHATSAPP_CLOUD_PHONE_NUMBER_ID	Meta phone number ID	Required
WHATSAPP_WEBHOOK_VERIFY_TOKEN	Webhook verification token	Required
APPOINTMENT_TEMPLATE_NAME	Approved appointment template name	Required

Variable	Purpose	Required
APPOINTMENT_TEMPLATE_PARAM_FORMAT	named or positional — must match Meta	Optional
MEDICINE_TEMPLATE_NAME	Medicine reminder template name	Optional
PRESCRIPTION_TEMPLATE_NAME	Prescription notification template	Optional

LiveKit Voice Agent

Variable	Purpose	Required
LIVEKIT_URL	LiveKit server WebSocket URL	Required
LIVEKIT_API_KEY	LiveKit API key	Required
LIVEKIT_API_SECRET	LiveKit API secret	Required
OPENAI_API_KEY	OpenAI API key (Realtime + GPT-4)	Required
SARVAM_API_KEY	Sarvam AI API key (STT + TTS)	Optional
SARVAM_STT_STREAMING	0 = batch REST (required for saaras:v3 model)	Optional
OPENAI_REALTIME_MODEL	Realtime model ID	Optional
LIVEKIT_QUEUE_ENABLED	Enable phone hold queue	Optional

Medicine Reminders

Variable	Purpose	Required
REMINDER_SYSTEM_ENABLED	Global kill-switch (set 0 to disable)	Optional
REMINDER_TIMEZONE	IANA timezone (default: Asia/Kolkata)	Optional
BREAKFAST_TIME	Reminder window HH:MM-HH:MM (default: 08:00)	Optional
LUNCH_TIME	Lunch window (default: 13:00-14:00)	Optional
DINNER_TIME	Dinner window (default: 20:00-21:00)	Optional
REMINDER_TEST_MODE	Fast test mode (1 = accelerated schedule)	Optional

Payments, Google & Email

Variable	Purpose	Required
RAZORPAY_KEY_ID	Razorpay public key	Required*

Variable	Purpose	Required
RAZORPAY_KEY_SECRET	Razorpay secret key	Required*
RAZORPAY_WEBHOOK_SECRET	Webhook HMAC secret	Required*
GOOGLE_CLIENT_ID	OAuth 2.0 client ID	Required*
GOOGLE_CLIENT_SECRET	OAuth 2.0 client secret	Required*
GOOGLE_REDIRECT_URI	OAuth callback URL	Required*
MAILTRAP_API_KEY	Mailtrap sending API key	Required*
MAIL_FROM	Sender email address	Required*
EXOTEL_ACCOUNT_SID	Exotel account SID	Required*
EXOTEL_API_KEY / TOKEN	Exotel API credentials	Required*

* Required only if that feature is used.

6. Database Documentation

MongoDB is the primary database (via Mongoose 8.x). All collections are scoped by hospitalId for multi-tenant data isolation. Redis is used exclusively for BullMQ job queues.

Collection: users

Authentication accounts for all portal users (hospital admins, doctors, tele-callers, super admins).

Field	Type	Required	Notes
_id	ObjectId	Auto	
name	String	Yes	Full display name
email	String	Yes	Unique across system
password	String	Yes	bcrypt hashed — never returned in API
role	Enum	Yes	user doctor tele_caller moderator hospital_admin admin super_admin
hospital	ObjectId → Hospital	No	Required for hospital-scoped roles
doctorProfile	ObjectId → Doctor	No	Sparse unique index
isActive	Boolean	Yes	Account enabled/disabled flag

Indexes: { role: 1 }, { doctorProfile: 1 } unique sparse

Collection: appointments

Core booking record linking patient, doctor, and hospital.

Field	Type	Notes
hospital	ObjectId → Hospital	
patient	ObjectId → Patient	Required
doctor	ObjectId → Doctor	Required
appointmentId	String	Display ID e.g. APT-0001
status	Enum	Today Upcoming Completed Cancelled
type	Enum	opd video_call phone_call
scheduledAt	Date	UTC ISO timestamp
notes	String	Doctor/agent notes
payment	ObjectId → PaymentHistory	Optional — links to Razorpay record

Indexes: Compound { hospital, patient, doctor, scheduledAt }

Collection: medicines

Master medicine catalog for prescription autocomplete and reminder targeting.

Field	Type	Required	Notes
medicineName	String	Yes	Prefix-searchable with case-insensitive collation index
type	String	Yes	Dosage form: Tablet, Capsule, Syrup, Vegicap, etc.
value	String	No	Numeric dose extracted from name (e.g. "500")
unit	String	No	Measurement unit: mg, ml, mcg, gm, IU — empty if not found

Indexes: { medicineName:1 }, { createdAt:-1 }, { medicineName:1 } collation {locale:"en",strength:2} named "medicineName_ci_prefix" — enables fast starts-with search

Collection: prescriptions

Digital prescriptions. The medicines[] array drives medicine reminder scheduling via BullMQ.

Field	Type	Notes
hospital / patient	ObjectId	References Hospital and Patient
appointment	ObjectId	Optional — links to appointment
date	Date	Prescription date
followUp	{ value, unit }	e.g. { value: 7, unit: "days" }
medicines[]	Array	name, dosage, frequency, duration, intake, meals {breakfast, lunch, dinner}, notes

All Collections Summary

Collection	Purpose
users	Auth accounts for all roles (admin, doctor, tele_caller, etc.)
refreshtokens	Active JWT refresh tokens with TTL auto-expiry index
hospitals	Root entity for multi-tenancy; all data scoped to hospital
doctors	Doctor profiles with availability hours and holiday date ranges
patients	Patient records: name, phone, age, gender, address
appointments	Booking records linking patient, doctor, hospital, slot
prescriptions	Digital prescriptions with medicine list and follow-up

Collection	Purpose
medicines	Master medicine catalog (300k+ records supported)
hospitalsettings	Per-hospital feature toggles (WhatsApp, tele-caller)
whatsapps	Meta WhatsApp Business Account credentials per hospital
whatsappplatformassignments	Maps hospitals to approved WhatsApp template names
paymenthistories	Razorpay payment capture records
paymenttransactions	Payment reconciliation (tele-caller + Razorpay sources)
payoutlists	Monthly payout summaries per hospital (draft → paid)
observations	Clinical notes per patient visit
exotelcalls	Synced Exotel call records for analytics (syncMonth indexed)
googleoauthtokens	Google OAuth 2.0 tokens per hospital (Calendar access)
whatsapponboardings	WhatsApp Business onboarding step tracking

7. API Documentation

Base URL: `https://<your-domain>/api`

Authentication: All protected endpoints require Authorization: Bearer <accessToken> in the request header.

Auth — /api/auth

Method	Route	Auth	Description
POST	/auth/register	Public	Register new user
POST	/auth/login	Public	Login; returns access + refresh tokens
POST	/auth/refresh	Public	Exchange refresh token for new access token
POST	/auth/logout	Public	Invalidate refresh token
POST	/auth/forgot-password	Public	Send password-reset email
POST	/auth/reset-password	Public	Reset password with email token
GET	/auth/me	Protected	Get current user profile
PATCH	/auth/me	Protected	Update profile name, email
POST	/auth/me/profile-picture	Protected	Upload profile picture
POST	/auth/logout-all	Protected	Revoke all refresh tokens for user

Login Request / Response

```
POST /api/auth/login
Body: { "email": "admin@hospital.com", "password": "secret" }
Response 200: { "success": true, "data": {
  "user": { "_id": "...", "name": "Admin", "role": "hospital_admin" },
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refreshToken": "eyJ..."
}
```

Core Resources

Method	Route	Role	Description
GET	/hospitals	Admin	List hospitals (paginated)
POST	/hospitals	Super Admin	Create hospital
GET	/doctors	Staff	List doctors (paginated)

Method	Route	Role	Description
GET	/doctors/names	Staff	Lightweight name+ID list
POST	/doctors	Admin	Create doctor
GET	/patients	Staff/TeleCaller	List patients
GET	/patients/:id/overview	Staff	Patient overview (appts + prescriptions)
POST	/patients	Admin	Create patient
GET	/appointments	Staff	List appointments (filterable)
POST	/appointments	Admin	Create appointment
PATCH	/appointments/:id	Staff	Update status, notes, etc.
GET	/prescriptions	Staff	List prescriptions
POST	/prescriptions	Staff	Create prescription

Medicines — /api/medicines

Method	Route	Auth	Description
GET	/medicines	Public	List all (paginated, filter by type)
GET	/medicines/search?q=	Public	Prefix search on medicineName — starts-with, case-insensitive
GET	/medicines/:id	Public	Get by ID
POST	/medicines	Protected	Create medicine
PATCH	/medicines/:id	Protected	Update medicine
DELETE	/medicines/:id	Protected	Delete medicine

Example: `GET /api/medicines/search?q=Para&limit;=10`

Returns only medicines whose medicineName starts with "Para" (Paracetamol, Paracip)

Does NOT return: Azithromycin, Dolo, or any non-prefix match

Other Routes

Route Prefix	Description
/api/whatsapp/webhook	Meta WhatsApp webhook (GET=verify, POST=receive messages)
/api/queue/status	Phone queue health check
/api/queue/webhook	LiveKit SIP webhook receiver
/api/livekit/token	Generate LiveKit room access tokens

Route Prefix	Description
/api/payments	Payment list and search (hospital-scoped)
/api/razorpay/webhook	Razorpay payment webhook (HMAC verified)
/api/dashboard	Hospital summary statistics
/api/exotel-calls	Exotel call records + sync trigger
/api/hospital-settings	Per-hospital feature toggles
/api/health	Health check — { "success": true, "app": "samvaad" }

8. Authentication & Authorization

Login Flow

Step	Action
1	Client sends POST /api/auth/login with email + password
2	Server verifies password against bcrypt hash in database
3	Server issues: accessToken (JWT, 15 min) + refreshToken (JWT, 7 days)
4	Client: accessToken in memory, refreshToken in HttpOnly cookie or secure storage
5	Every API request: Authorization: Bearer
6	On 401: client calls POST /api/auth/refresh to get a new accessToken
7	Logout: POST /api/auth/logout deletes refreshToken from refreshtokens collection
8	Logout-all: deletes ALL refresh tokens for the user (all devices signed out)

Role Hierarchy (Lowest to Highest)

user < doctor < tele_caller < moderator < hospital_admin < admin < super_admin

Permission Map

Role	Capability
super_admin	All operations, cross-hospital access
admin	All data within assigned hospital
hospital_admin	Hospital data management (own hospital only)
moderator	Read/write most data; no delete permissions
doctor	Own hospital data; create/view prescriptions
tele_caller	Patient list; limited booking actions
user	Minimal access

Security Measures

- Passwords hashed with bcryptjs (never stored in plaintext)
- JWT access tokens expire in 15 minutes; refresh tokens 7 days
- Refresh tokens stored in MongoDB with TTL auto-expiry index
- Rate limiting applied globally via express-rate-limit
- All :id parameters validated as MongoDB ObjectId before DB queries

- CORS restricted to configured allowed origins
- Razorpay webhooks verified with HMAC-SHA256 signature
- WhatsApp webhooks verified with configured verify token
- Google OAuth state parameter signed with HMAC to prevent CSRF

9. Third-Party Integrations

1. LiveKit

Property	Detail
Purpose	WebRTC/SIP infrastructure for voice agent and phone queue
SDK	@livekit/agents v1.2.x, livekit-server-sdk v2.15.x
Credentials	LIVEKIT_URL, LIVEKIT_API_KEY, LIVEKIT_API_SECRET
Setup	LiveKit Cloud → create project → configure SIP trunk → point Exotel to LiveKit SIP URI
Failure Handling	websocketErrorGuard.js patches uncaught WS errors; PM2 autorestart: true

2. OpenAI

Property	Detail
Purpose	Realtime voice conversation (gpt-realtime-mini) + post-call extraction (GPT-4.1)
SDK	openai ^6.21.0, @livekit/agents-plugin-openai
Credentials	OPENAI_API_KEY
Models	OPENAI_REALTIME_MODEL (default: gpt-realtime-mini-2025-12-15), OPENAI_LLM_MODEL (default: gpt-4.1)
Failure Handling	Tool errors return descriptive messages; post-call pipeline wrapped in try/catch

3. Sarvam AI

Property	Detail
Purpose	Hindi + English speech-to-text and text-to-speech
SDK	sarvamai ^1.1.7
Credentials	SARVAM_API_KEY
Key Setting	SARVAM_STT_STREAMING=0 — batch REST required (streaming WS rejects saaras:v3 model)
Fallback	Falls back to OpenAI STT if SARVAM_API_KEY is not set

4. Meta WhatsApp Cloud API

Property	Detail
Purpose	Appointment confirmations, prescription links, medicine reminders

Property	Detail
Credentials	WHATSAPP_CLOUD_ACCESS_TOKEN, WHATSAPP_CLOUD_PHONE_NUMBER_ID
Setup	Meta Business Suite → WhatsApp App → permanent token → webhook → submit templates for approval
Template Format	APPOINTMENT_TEMPLATE_PARAM_FORMAT=named or positional must match Meta creation
Failure Handling	BullMQ: 5 retries, exponential backoff; surfaces error_data.details for debugging

5. Razorpay

Property	Detail
Purpose	Online payment collection from patients
SDK	razorpay ^2.9.6
Credentials	RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET
Setup	Razorpay Dashboard → API Keys → Webhooks → add /api/razorpay/webhook endpoint
Failure Handling	Webhook HMAC verified; payments stored as pending until captured event

6. Google Calendar

Property	Detail
Purpose	Create Google Meet video appointments; sync to doctor calendars
Credentials	GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GOOGLE_REDIRECT_URI
Setup	GCP Console → Enable Calendar API → OAuth 2.0 credentials → admin authorizes via /api/public/google-auth
Storage	Tokens stored in googleoauthtokens collection; auto-refreshed by googleapis library

7. Exotel

Property	Detail
Purpose	SIP telephony (inbound calls to voice agent) + call analytics
Credentials	EXOTEL_ACCOUNT_SID, EXOTEL_API_KEY, EXOTEL_API_TOKEN
Setup	Exotel account → ExoPhone → configure routing to LiveKit SIP URI
Analytics	Monthly cron syncs call records: POST /api/exotel-calls/sync/current-month

8. Mailtrap

Property	Detail
Purpose	Transactional email: password reset, welcome emails
Credentials	MAILTRAP_API_KEY (Bearer token), MAIL_FROM (sender address)
Setup	Mailtrap → Sending → Domains → verify domain → API tokens → copy key to env

10. Deployment Guide

Development Setup

```
# 1. Clone repository
git clone <repo-url> && cd samvaad-server

# 2. Install dependencies
npm install

# 3. Configure environment
cp .env.example .env # then edit with your credentials

# 4. Ensure MongoDB + Redis are running locally

# 5. Start API (all workers embedded)
npm run dev

# 6. Start voice agent (separate terminal)
npm run livekit:dev

# 7. API only (no voice workers)
npm run dev:api-only
```

Production Deployment (VPS)

```
# 1. SSH into VPS
ssh root@<droplet-ip>

# 2. Pull latest code
cd /path/to/samvaad-server && git pull origin main

# 3. Install/update dependencies
npm ci

# 4. Start all PM2 processes
pm2 start ecosystem.config.cjs

# 5. Save PM2 list (survives server reboots)
pm2 save && pm2 startup
```

PM2 Process Reference

PM2 Name	Script	Port	Purpose
samvaad-api	index.js	3000	REST API server
samvaad-livekit-worker	livekit-agent/main.js	8081	Voice booking agent
samvaad-queue-worker	livekit-agent/phoneQueueWorker.js	8082	Phone hold queue
samvaad-reminder-worker	src/workers/reminderWorkerEntry.js	—	Medicine reminders (BullMQ)

Rollback Procedure

```
# Identify last working commit
git log --oneline -10

# Checkout previous commit
git checkout <commit-hash>

# Reinstall dependencies and restart
npm ci && pm2 restart all
```


11. Infrastructure Documentation

Component	Detail
Hosting	Linux VPS — DigitalOcean Droplet
Process Manager	PM2 (4 processes, autorestart, max_restarts: 20)
Timezone	Asia/Kolkata (IST) enforced in all PM2 processes via TZ env
Reverse Proxy	Not Found in Codebase — Nginx/Caddy assumed upstream
SSL/TLS	Not Found in Codebase — upstream proxy expected
File Storage	Local disk at ./uploads (configurable via UPLOADS_ROOT)
CI/CD	Not Found in Codebase — manual git pull + pm2 restart

Port Assignments

Port	Service	Notes
3000	REST API (Express)	Main HTTP port; proxied through Nginx/Caddy
8081	LiveKit Worker (Health)	Do NOT run queue worker on this port
8082	Queue Worker (Health)	Explicitly set to avoid EADDRINUSE conflict with port 8081
6379	Redis	BullMQ job queues
27017	MongoDB	Standard port (Atlas uses SRV connection string)

12. Monitoring & Logging

PM2 Log Commands

```
pm2 logs # tail all processes
pm2 logs samvaad-api # API server only
pm2 logs samvaad-livekit-worker # voice agent
pm2 logs --lines 200 samvaad-api # last 200 lines

# Log files on disk:
~/.pm2/logs/samvaad-api-out.log # stdout
~/.pm2/logs/samvaad-api-error.log # stderr
```

Log Level Control

Setting	Effect
LOG_LEVEL=warn (production default)	Only warnings and errors — reduces LiveKit framework noise
LOG_LEVEL=info or debug (dev)	Full verbose logging
AGENT_VERBOSE_CONNECTION_LOGS=1	Detailed agent startup banner (auto-on in non-production)

Monitoring

```
pm2 monit # Real-time CPU/memory dashboard
pm2 status # Process status table
```

No external APM (Datadog, New Relic) or error tracking (Sentry) is currently configured. All errors are logged to PM2 log files only. Adding Sentry is strongly recommended — see Section 18.

13. Testing

No automated test suite exists. npm test exits with error. This is a known gap — see Section 18 for recommendations.

Manual Testing Procedures

API Health Check

```
curl https://<your-domain>/api/health
# Expected: { "success": true, "app": "samvaad", "timestamp": "..."} }
```

Voice Agent

Step	Action
1	Call the configured Exotel ExoPhone number
2	Agent answers in English/Hindi within 5 seconds
3	Say "I want to book an appointment"
4	Agent asks for doctor preference and desired time
5	Confirm appointment; agent speaks confirmation with appointment ID
6	Check patient WhatsApp for appointment confirmation message

Medicine Reminders (Test Mode)

```
# In .env: REMINDER_TEST_MODE=1
pm2 restart samvaad-reminder-worker
pm2 logs samvaad-reminder-worker
# Breakfast fires at +2min, lunch +25min, dinner +48min
```

14. Known Issues

Issue	Severity	Notes
No automated test suite	High	npm test exits with error — zero test coverage
PM2 env override	Medium	dotenv does not override PM2 env; must use pm2 delete + pm2 start --update-env
Sarvam STT streaming	Medium	saaras:v3 rejected by streaming WS; keep SARVAM_STT_STREAMING=0
WhatsApp template format	Medium	APPOINTMENT_TEMPLATE_PARAM_FORMAT must exactly match Meta template creation
No reverse proxy config	Low	SSL termination assumed but not in repo
No CI/CD pipeline	Low	Manual deploy via git pull + pm2 restart
No error tracking	Low	Errors only visible in PM2 logs

15. Backup & Recovery

Database Backup

```
# MongoDB Atlas: enable automated backups in Atlas UI (recommended)

# Self-hosted — manual backup:
mongodump --uri="$MONGODB_URI" --out=/backup/$(date +%Y-%m-%d)
tar -czf /backup/samvaad-$(date +%Y-%m-%d).tar.gz /backup/$(date +%Y-%m-%d)
```

Restore Process

```
# Full restore:
mongorestore --uri="$MONGODB_URI" /backup/2026-06-23/

# Single collection restore:
mongorestore --uri="$MONGODB_URI" --db samvaad \
--collection appointments /backup/2026-06-23/samvaad/appointments.bson
```

Disaster Recovery — Fresh VPS

```
# 1. Install Node.js >=18, npm, PM2, Redis
npm install -g pm2

# 2. Clone repository and restore .env
git clone <repo-url> /opt/samvaad-server && cd /opt/samvaad-server

# 3. Install dependencies and start
npm ci
pm2 start ecosystem.config.cjs && pm2 save && pm2 startup

# 4. Restore MongoDB
mongorestore --uri="$MONGODB_URI" /backup/latest/
```

16. Security Considerations

Sensitive Data Areas

Area	Risk	Mitigation
.env file	Contains ALL secrets and API keys	Never commit to git; store encrypted externally
JWT secrets	Default dev values are insecure	Change JWT_ACCESS_SECRET and JWT_REFRESH_SECRET in production
WhatsApp access token	Permanent Meta token	Rotate if compromised; store only in .env
OpenAI API key	Unlimited billing risk if leaked	Set spend limits in OpenAI dashboard; rotate quarterly
Razorpay live keys	Financial risk	Use test keys in dev; never commit live keys to git
MongoDB URI	Full database access	Atlas IP allowlist; dedicated DB user with minimal permissions
Patient data	Healthcare sensitivity (PHI)	Restrict DB access; consider encryption at rest

Recommended Improvements

- Add automated test suite with auth and data isolation coverage
- Configure Nginx/Caddy with SSL/TLS termination and HSTS headers
- Integrate Sentry or similar for real-time error alerting
- Enable MongoDB Atlas IP allowlist — restrict to VPS IP only
- Set up quarterly secret rotation for JWT secrets and API keys
- Add audit logging — track who created/modified appointments
- Rate limit per hospital (not just globally)

17. Operational Runbook

Deploy Code Update

```
git pull origin main
npm ci
pm2 restart all
pm2 status # verify all online
```

Update Environment Variables

CRITICAL: PM2 ecosystem env overrides .env. A plain pm2 restart does NOT pick up .env changes. You must delete and re-start the process.

```
nano .env # make your changes

pm2 delete samvaad-api
pm2 start ecosystem.config.cjs --only samvaad-api
```

Rotate JWT Secrets

```
# Generate new secrets:
node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"

# Update JWT_ACCESS_SECRET + JWT_REFRESH_SECRET in .env
# NOTE: This logs out ALL users – plan a maintenance window
pm2 delete samvaad-api
pm2 start ecosystem.config.cjs --only samvaad-api
```

Bulk Import Medicines

```
# Dry run first:
node scripts/import-medicines/import-medicines.js --file=/path/to/medicines.xlsx --dry-run

# Full import:
node scripts/import-medicines/import-medicines.js --file=/path/to/medicines.xlsx --batch=1000
```

18. Future Enhancements

Priority	Enhancement	Notes
High	Automated test suite	Unit + integration tests; currently zero coverage
High	CI/CD pipeline	GitHub Actions: test → lint → deploy on merge to main
High	Error tracking (Sentry)	Real-time alerting for production errors
Medium	Multi-language voice agent	Extend beyond Hindi/English: Tamil, Telugu, Kannada
Medium	SMS fallback	When WhatsApp delivery fails, fall back to Exotel SMS
Medium	Doctor mobile app	Doctors manage their own schedules from mobile
Medium	Patient portal	Self-service: view prescriptions + appointment history
Medium	Healthcare compliance	Encryption at rest, audit logs, access reviews
Low	Docker containerization	Dockerfile + docker-compose for reproducible deploys
Low	Horizontal scaling	Redis-backed sessions + PM2 cluster mode
Low	Analytics dashboard	Call volume, booking conversion, reminder delivery rate

19. Appendix

Glossary

Term	Definition
STT	Speech-to-Text — converts caller audio to text (Sarvam AI)
TTS	Text-to-Speech — converts agent response to audio (Sarvam AI)
VAD	Voice Activity Detection — detects when caller is speaking (Silero)
SIP	Session Initiation Protocol — telephony used by Exotel + LiveKit
BullMQ	Redis-backed job queue library for Node.js
IST	Indian Standard Time (Asia/Kolkata, UTC+5:30)
WABA	WhatsApp Business Account
OPD	Out-Patient Department (appointment type)
PM2	Production Process Manager for Node.js
LiveKit	Open-source WebRTC infrastructure for real-time audio/video
HMAC	Hash-based Message Authentication Code — webhook signature verification
ODM	Object Document Mapper — Mongoose maps JS objects to MongoDB documents

External References

Resource	URL
LiveKit Agents SDK	https://docs.livekit.io/agents/
OpenAI Realtime API	https://platform.openai.com/docs/guides/realtime
Sarvam AI Docs	https://docs.sarvam.ai
Meta WhatsApp Cloud API	https://developers.facebook.com/docs/whatsapp/cloud-api
Razorpay API Docs	https://razorpay.com/docs/api
BullMQ Docs	https://docs.bullmq.io
Mongoose Docs	https://mongoosejs.com/docs
PM2 Docs	https://pm2.keymetrics.io/docs

Contact Information

Not Found in Codebase — contact details to be provided by the project owner.

End of Document

Version 1.0.0 | Samvaad AI | June 23, 2026 | Confidential